

El algoritmo de programación dinámica para el problema del knapsack se puede implementar de dos maneras: iterativa o recursiva.

La implementación iterativa es la siguiente:

En la implementación recursiva, se define una función que devuelve el valor máximo de la mochila para un tamaño dado de la mochila y un conjunto de objetos.

Contenido

- 1. Introducción.
- 2. The Knapsack Problem.
- 3. Solución con Programación Dinámica.
- 4. Ejemplo del Itinerario de Viaje.
- 5. Ejemplo de Bari.
- 6. Análisis de Complejidad.
- 7. Resumen.
- 8. Ejercicios.

— 1. Introducción.

Vamos a resolver el problema del “knapsack” utilizando una técnica conocida como “dynamic programming”, en español programación dinámica.

La programación dinámica es una técnica que busca resolver un problema al dividirlo en etapas de decisión. Cada decisión dependerá de la decisión anterior.

Esta técnica tiene una parte de arte y ciencia, pues para encontrar la fórmula de recurrencia matemática se debe conocer bien la descripción del problema.

— 2. The Knapsack Problem.

Volvamos a describir el problema del Knapsack.

Suponga que tenemos una lista con los siguientes artículos.

Ide.	Objeto	Ganancia (\$)	Peso (kilos)
g	guitarra	1500	1
s	stereo	3000	4
l	laptop	2000	3
i	iphone	2000	1

>>> Modelaje de los Datos.

En la lista se menciona el identificador, la ganancia de vender el artículo y su peso. Se utiliza una lista con la siguiente descripción:

```
[ nombre, [ganancia_$, peso_kilos]]
```

Vamos a suponer adicionalmente, que los pesos de los artículos son valores enteros.

Tenemos entonces:

```
objetos:
[
[ g,    [1500, 1]],
[ s,    [3000, 4]],
[ l,    [2000, 3]],
[ i,    [2000, 1]]
];
```

Supongamos que tenemos una mochila que puede llevar 4 kilos de peso.

El objetivo es escoger de esta lista de artículos, aquellos que van a producir el máximo valor al sumar sus ganancias con un peso menor o igual a 4 kilos.

>>> Matemáticamente la fórmula que se utiliza es:

Si el objeto no cabe en la mochila:

$$\text{mat}[i][j] = \max \begin{cases} \text{mat}[i-1][j] \\ 0 \end{cases}$$

Si el objeto si cabe en la mochila:

$$\begin{cases} \text{mat}[i-1][j] \\ \text{mat}[i-1][j-1] + \text{ganancia}_i \end{cases}$$

$$\text{mat}[i][j] = \max \begin{cases} \text{gananciaDelObjeto} \\ + \text{mat}[i-1][j - \text{pesoDelObjeto}] \end{cases}$$

3. Solución con Programación Dinámica.

Primero creamos una tabla con los posibles pesos y los artículos. Por lo tanto el tamaño de la tabla es (num_artículos * peso). En este caso se han indicado los índices de la fila que son los artículos y los índices de la columna corresponden al peso. El peso está dado en unidades enteras.

Obj	1k	2k	3k	4k
g	--	--	--	--
s	--	--	--	--
l	--	--	--	--
i	--	--	--	--

Vamos a ir llenando esta matriz por filas. Una vez que se hayan llenado todas la filas, vamos a tener la solución del problema.

>>> Llenando la fila 1.

La guitarra cabe en una mochila de 1k, de 2k, de 3k y de 4k.
Se anota la ganancia en la fila.
[g, [1500, 1]]

Obj	1k	2k	3k	4k
g	1500 g	1500 g	1500 g	1500 g
s	--	--	--	--
l	--	--	--	--
i	--	--	--	--

>>> Llenando la fila 2.

```
[ g, [1500, 1]]
[ s, [3000, 4]]
```

Obj	1k	2k	3k	4k
g	1500 g	1500 g	1500 g	1500 g
s	1500 g	1500 g	1500 g	3000 s
l	--	--	--	--
i	--	--	--	--

>>> Llenando la fila 3.

Hagamos lo mismo con la fila del laptop.

```
[ g, [1500, 1]]
[ s, [3000, 4]]
[ l, [2000, 3]]
```

Obj	1k	2k	3k	4k
g	1500 g	1500 g	1500 g	1500 g
s	1500 g	1500 g	1500 g	3000 s
l	1500 g	1500 g	2000 l	3500 gl ◀◀
i	--	--	--	--

Observemos la decisión tomada en el campo marcado.

Se puede poner un stereo de 4k, y ganar \$3000.

o

Se puede poner un laptop de 3k, ganar \$2000, queda 1k y ahí poner una guitarra de 1k, ganar \$1500, en total $\$2000 + \$1500 = \$3500$

Por esta razón se calcularon los valores de mochilas más pequeñas.

Puede revisar que hemos usado la fórmula descrita anteriormente en todos los casos.

>>> Llenando la fila 4.

```
[ g, [1500, 1]]
[ s, [3000, 4]]
[ l, [2000, 3]]
[ i, [2000, 1]]
```

i	1k	2k	3k	4k
g	1500 g	1500 g	1500 g	1500 g
s	1500 g	1500 g	1500 g	3000 s
l	1500 g	1500 g	2000 l	3500 gl
i	2000 i	3500 gi	3500 gi	4000 li

●● Algunas observaciones.

- Conforme aumenta el tamaño de las filas, aumenta el valor de la ganancia.
- No es importante el orden de los objetos en las filas, siempre se llega a la misma respuesta.
- Si tuviera un collar que pesa 0.50k y una ganancia \$2500. Se podría resolver el problema, pero tendríamos que usar columnas, o pesos con incrementos de 0.50. Esto produciría una matriz de un gran tamaño. En este capítulo, no trataremos este caso.

— 4. Ejemplo del Itinerario de Viaje.

Vamos de vacaciones para Londres. Tenemos 4 días para pasear y conocer lugares. Hay muchas cosas que usted desea hacer, pero no se pueden visitar todos los lugares ni hacer todas las cosas que uno desea en tan poco tiempo.

Hemos hecho una lista de los lugares que nos gustaría visitar, del tiempo que se necesita para ver cada lugar y del nivel de satisfacción que nos produce.

Ide.	Atracción	Rating	Tiempo
w	westminsterAbbey	70	1 día
g	globeTheater	60	1 día
n	nationalGallery	90	2 días
b	britishMuseum	90	4 días
s	stPaulsCathedral	80	1 día

Con base en esta información, se desea escoger aquellas atracciones que tengan el mayor rating posible dentro del número de días disponibles.

●● Modelaje de los Datos.

Vamos a representar los lugares como una lista:

```
objetos:
[
[ w, [70, 1]],
[ g, [60, 1]],
[ n, [90, 2]],
[ b, [90, 4]],
[ s, [80, 1]]
];
```

Y resolveremos este problema mediante programación dinámica. Es fácil descubrir que es un problema de knapsack. En lugar de ganancias tenemos ratings, y en lugar del peso de la mochila tenemos los días que se necesitan para visitar cada lugar.

●● Tabla de Soluciones.

```
objetos:
[
[ w, [70, 1]],
[ g, [60, 1]],
[ n, [90, 2]],
[ b, [90, 4]],
[ s, [80, 1]]
];
```

Aquí pueden ver la tabla de soluciones que se produce:

b	0 []	10 a	20 b	20 b	30 ab	30 ab	30 ab	30 ab
c	0 []	10 a	20 b	50 c	50 c	60 ac	70 bc	70 bc
d	0 []	10 a	20 b	50 c	55 d	60 ac	70 bc	75 bd

Por lo que en la mochila se debe poner los objetos “b” y “d”, con una ganancia de \$75.

6. Análisis de Complejidad.

El algoritmo original exhaustivo tiene un tiempo de ejecución de $O(2^n)$.

El utilizar el algoritmo de programación dinámica, el tiempo que se requiere para solucionar el problema, es equivalente al tiempo requerido para llenar la matriz.

Si tenemos “n” objetos, y el peso de la mochila es un valor “w”, el tamaño de la matriz será de: $n*w$, por lo tanto el tiempo de ejecución está dado por:

$O(n*w)$

7. Resumen.

- La programación dinámica es útil cuando se está tratando de optimizar un problema sujeto a restricciones.
- Se puede utilizar cuando el problema se puede descomponer en problemas más pequeños.
- Todo problema de programación lineal necesita una matriz de cálculos, ya sea implícita o explícita.
- Los valores en las celdas son usualmente lo que se está tratando de maximizar o minimizar.
- Cada celda representa un subproblema.
- Para diferentes problemas de programación lineal, se deben utilizar diferentes fórmulas para calcular la matriz.

8. Ejercicios.

●● Ejercicio 1.

Se tiene este nuevo knapsack con una mochila de 4kilos.

objetos:

```
[  
[ guitarra, [1500, 1]],  
[ stereo,   [3000, 4]],  
[ laptop,   [2000, 3]],  
[ iphone,   [2000, 1]],  
[ mxp,      [1000, 1]]  
];
```

Cuál es la solución de este problema?

●● Ejercicio 2.

Para esta lista de objetos, puede crear el árbol que ejecuta la solución recursiva.

```
[objetos:  
[ [c, [10, 1]],  
  [b, [10, 1]],  
  [a, [10, 1]]  
]
```

●● Ejercicio 3.

Hay alguna manera que la solución recursiva indique los objetos que van en la mochila?

Puede usar una instrucción print, o una lista que guarde los objetos.

●● Ejercicio 4.

Supongamos que vamos de campamento. Tenemos una mochila en la que caben 6 kilos. Y se desean llevar algunos de los siguientes objetos. Para cada objeto se muestra su valor y su peso.

- agua, \$10, 3k
- libro, \$ 3, 1k
- comida, \$ 9, 2k
- jacket, \$ 5, 2k
- cámara, \$ 6, 1k

Cuál es el conjunto objetos según el algoritmo óptimo que se deben llevar en la mochila para maximizar su valor?

Cuál es el conjunto de objetos según el algoritmo greedy que se deben llevar en la mochila para maximizar su valor?

●● Ejercicio 5.

Para un problema de Knapsack, donde existe una cantidad infinita de cada uno de los objetos, investigue:

Cómo se construye un algoritmo óptimo?

Cómo se construye un algoritmo greedy?